Práctica N°4: Máquinas de Estados Finitos (FSM)

Índice

1. Objetivos **3**
- 1.1. Objetivo general . . . 3
- 1.2. Objetivos específicos . . . 3

2. Marco Teórico. **3**
- 2.1. Máquinas de estados . . . 3
  - 2.1.1. Maquina de Moore . . . 4
  - 2.1.2. Maquina de Mealy . . . 5

3. GPIO Externos en la Nexys A7-100T **6**
- 3.1. Distribución de los Pmod Ports . . . 6
- 3.2. Usos de los Pmod Ports . . . 7
- 3.3. Referencia Visual . . . 7

4. Descripción del circuito en VHDL **8**
- 4.1. Entidad y puertos . . . 14
- 4.2. Tipos de datos y estados de la FSM . . . 15
- 4.3. Divisores de reloj y generación de 1 Hz . . . 15
- 4.4. Sincronización, *debounce* y *one-shot* de `btn_dec5` . . . 16
- 4.5. Registro de estado y contador con prioridad al botón . . . 18
- 4.6. Lógica de transición (`next_state`) . . . 19
- 4.7. Asignación de salidas (LEDs por eje) . . . 19
- 4.8. Decodificador BCD a 7 segmentos (activo en bajo) . . . 20
- 4.9. Cálculo de dígitos (decenas y unidades) . . . 21
- 4.10. Multiplexado de 8 dígitos . . . 21
- 4.11. Patrón universal para cualquier máquina de estados . . . 22
- 4.12. Diagrama de bloques . . . 22
- 4.13. Mapeo de pines y niveles eléctricos (Nexys A7) . . . 22

5. Resultados en la placa Nexys **24**

1. Objetivos

1.1. Objetivo general

Diseñar e implementar en `VHDL` un controlador de semáforo con visualización de cuenta regresiva en displays de 7 segmentos para la tarjeta Nexys A7-100T, que emplee una máquina de estados finitos (FSM) síncrona, multiplexado de displays y tratamiento robusto de entradas mediante *debounce* y *one-shot*.

1.2. Objetivos específicos

- Modelar la lógica del semáforo como una FSM con estados y transiciones claramente definidas.
- Implementar divisores de reloj para generar un *tick* de 1 Hz y una frecuencia de refresco adecuada para el multiplexado de 8 dígitos.
- Integrar un pulsador con sincronización a reloj, *debounce* por tiempo y un pulso único (*one-shot*) para accionar la reducción de tiempo o el salto de estado.
- Desarrollar un decodificador BCD a 7 segmentos *activo en bajo* y una lógica de multiplexado estable para dos dígitos de conteo.
- Validar la coherencia de las salidas de LEDs por ejes (N/S, E/O) y la sincronía entre la visualización y la FSM.

2. Marco Teórico.

2.1. Máquinas de estados

Una máquina de estados finita (MEF), también conocida como autómata finito, es un modelo matemático utilizado para describir y representar sistemas que exhiben diferentes estados y transiciones entre ellos. Se utiliza en el campo de la computación y en la ingeniería para diseñar y analizar sistemas de control, protocolos de comunicación y lógica secuencial.

Una MEF consta de un conjunto finito de estados, donde cada estado representa una condición o configuración específica del sistema en un momento dado. Puede haber un estado inicial que representa el estado inicial del sistema y uno o más estados finales (o de aceptación) que indican que el

sistema ha alcanzado un estado deseado o ha completado una secuencia de eventos.

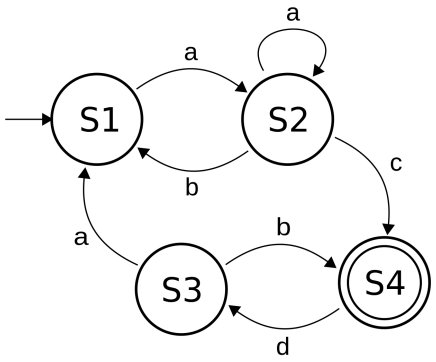


Figura 1: Diagrama de una maquina de estados finitos

Las Transiciones de un estado a otro se generan en respuesta a eventos de entrada externos e internos; a su vez estas transiciones y/o subsecuentes estados pueden generar otros eventos de salida. Se pueden representar mediante una tabla de transiciones, un diagrama de estados o mediante funciones de transición. [**?**]

La implementación de una máquina de estados finita no es una técnica única, sino una variedad de métodos que comparten un principio común. En un nivel alto, existen dos formas de implementar una máquina de estados finita (se hablará de ellas más adelante). La primera opción implica el uso de una estructura if-else. Esta estrategia es fácil de entender e implementar, pero tiene desventajas significativas, el código se vuelve difícil de comprender ya que todo se agrupa en una única condición.

El segundo método para implementar una MEF es utilizar una estructura switch. Esta permite escribir una función separada para cada estado y luego ejecutar una de estas funciones en función del estado actual. La ventaja de utilizar un switch es que facilita comprender el flujo de ejecución a través del código. Sin embargo, esta implementación no es muy escalable.

2.1.1. Maquina de Moore

Una máquina de Moore es un tipo de máquina en la cual las salidas dependen únicamente del estado actual en el que se encuentra la máquina. Es decir, las salidas están asociadas directamente con los estados y no se ven influenciadas por las entradas del sistema.

En una máquina de Moore, cada estado tiene asociada una salida específica que se mantiene constante mientras la máquina se encuentre en ese estado.

Cuando la máquina realiza una transición de estado, la salida puede cambiar según la nueva configuración. El cambio en las salidas ocurre únicamente cuando se produce una transición de estado y no está determinado por las entradas del sistema. [**?**]

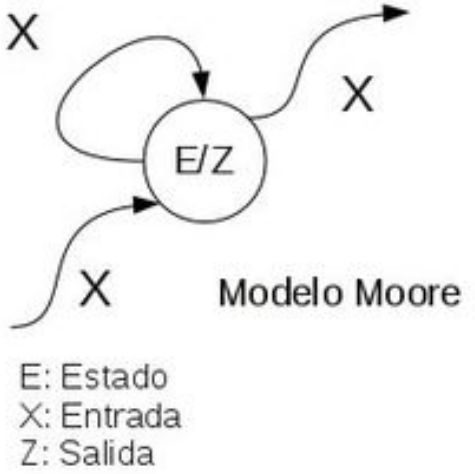


Figura 2: Diagrama de una maquina de Moore

2.1.2. Maquina de Mealy

una Máquina de Mealy es un tipo de máquina de estados finitos que genera una salida basándose en su estado actual y una entrada. Esto significa que el Diagrama de estados incluirá ambas señales de entrada y salida para cada línea de transición.

En una máquina de Mealy, la salida puede cambiar instantáneamente cuando se produce una transición de estado y las entradas del sistema también cambian. Esto significa que las salidas pueden ser variables y dependen tanto del estado actual como de las entradas actuales. Por lo tanto, la respuesta del sistema en una máquina de Mealy puede ser más inmediata y sensible a los cambios en las entradas. [**?**]

Las máquinas de Mealy se utilizan en aplicaciones donde las salidas deben ser sensibles a las entradas del sistema y el comportamiento del sistema puede cambiar rápidamente en función de los cambios en las entradas.

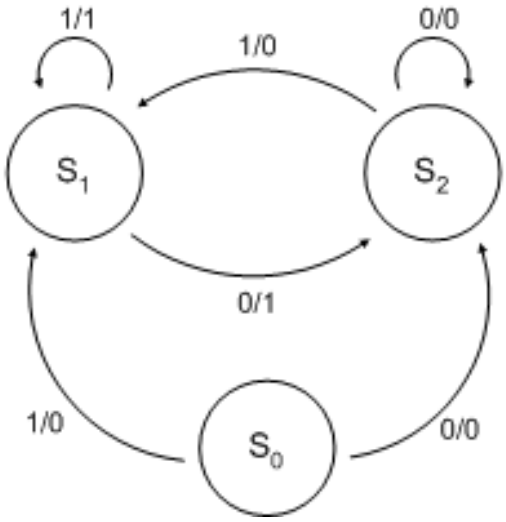


Figura 3: Diagrama de una maquina de Mealy

3. GPIO Externos en la Nexys A7-100T

Una de las características más útiles de la tarjeta **Nexys A7-100T** es la presencia de conectores de propósito general (GPIO) accesibles para el usuario. Estos pines están agrupados en los encabezados denominados **Pmod Ports**, identificados como **JA, JB, JC y JD**. Cada uno de estos puertos permite conectar dispositivos externos tales como sensores, actuadores o módulos de comunicación.

3.1. Distribución de los Pmod Ports

Cada conector Pmod está compuesto por **8 señales de entrada/salida digitales**, organizadas en dos grupos de 4 pines. Además, cuentan con líneas de alimentación y tierra que permiten energizar dispositivos externos:

- **8 pines GPIO**: programables desde el diseño en VHDL o Verilog.
- **2 pin de 3.3V fijo**: salida de tensión estable para alimentar periféricos externos.
- **2 pin de GND**: referencia común de tierra.

En total, cada puerto Pmod dispone de 12 pines (8 de propósito general, 2 de 3.3V, 2 de GND. Los niveles de tensión están limitados a **3.3V**, por lo que cualquier dispositivo conectado debe ser compatible con este estándar de lógica.

3.2. Usos de los Pmod Ports

Gracias a estos conectores, el estudiante puede extender las capacidades de la tarjeta de manera sencilla:

- Conectar **módulos de sensores** (temperatura, luz, acelerómetros, etc.).
- Interconectar con **módulos de comunicación** (UART, SPI, I2C).
- Manejar **actuadores externos** (displays, relés, motores con driver).

De esta manera, la Nexys A7 se convierte en una plataforma flexible tanto para prácticas educativas como para prototipado de proyectos.

3.3. Referencia Visual

En la Figura 4 se muestra la disposición de los conectores Pmod JA, JB, JC y JD en la tarjeta, tomada del manual oficial de Digilent. Esto permitirá al estudiante identificar rápidamente qué puerto usar de acuerdo con el dispositivo externo que desee conectar.



Figure 10.1 Pmod Connectors; Front View, as Loaded on PCB

Table 10.1. Nexys A7 Pmod pin assignments.

Pmod JA	Pmod JB	Pmod JC	Pmod JD	Pmod XADC
JA1: C17	JB1: D14	JC1: K1	JD1: H4	JXADC1: A13 (AD3P)
JA2: D18	JB2: F16	JC2: F6	JD2: H1	JXADC2: A15 (AD10P)
JA3: E18	JB3: G16	JC3: J2	JD3: G1	JXADC3: B16 (AD2P)
JA4: G17	JB4: H14	JC4: G6	JD4: G3	JXADC4: B18 (AD11P)
JA7: D17	JB7: E16	JC7: E7	JD7: H2	JXADC7: A14 (AD3N)
JA8: E17	JB8: F13	JC8: J3	JD8: G4	JXADC8: A16 (AD10N)
JA9: F18	JB9: G13	JC9: J4	JD9: G2	JXADC9: B17 (AD2N)
JA10: G18	JB10: H16	JC10: E6	JD10: F3	JXADC10: A18 (AD11N)

Figura 4: Conectores Pmod JA, JB, JC y JD de la Nexys A7-100T.

4. Descripción del circuito en VHDL

Listado de código ()

```
1  library IEEE;
2  use IEEE.STD_LOGIC_1164.ALL;
3  use IEEE.NUMERIC_STD.ALL;
4
5  entity semaforo_dia is
6      Port (
7          clk   : in  STD_LOGIC;                -- 100 MHz
               Nexys A7
8          reset : in  STD_LOGIC;                -- Activo en
               alto
9          btn_dec5 : in STD_LOGIC;              -- Boton:
               restar 5 s / saltar estado
10
11         -- 7-seg multiplexados (activo en bajo)
12         seg : out STD_LOGIC_VECTOR(0 to 6); -- CA..CG
13         an  : out STD_LOGIC_VECTOR(7 downto 0); -- AN7..AN0 (
               activo en bajo)
14
15         -- LEDs Norte
16         led_n_rojo     : out STD_LOGIC;
17         led_n_amarillo : out STD_LOGIC;
18         led_n_verde    : out STD_LOGIC;
19
20         -- LEDs Sur
21         led_s_rojo     : out STD_LOGIC;
22         led_s_amarillo : out STD_LOGIC;
23         led_s_verde    : out STD_LOGIC;
24
25         -- LEDs Este
26         led_e_rojo     : out STD_LOGIC;
27         led_e_amarillo : out STD_LOGIC;
28         led_e_verde    : out STD_LOGIC;
29
30         -- LEDs Oeste
31         led_o_rojo     : out STD_LOGIC;
32         led_o_amarillo : out STD_LOGIC;
33         led_o_verde    : out STD_LOGIC
34     );
35 end semaforo_dia;
36
37 architecture Behavioral of semaforo_dia is
38
39     -- ======= FSM semaforo =======
```

```
40      type state_type is (NS_VERDE, NS_AMARILLO, EO_VERDE,
            EO_AMARILLO);
41      signal current_state, next_state : state_type := NS_VERDE
            ;
42
43      signal counter : integer range 0 to 10 := 10;
44
45      -- ======= Relojes / divisores =======
46      constant CLK_FREQ_HZ          : integer := 100_000_000;
            -- Nexys A7
47      -- Tick 1 Hz:
48      constant ONE_HZ_DIV_MAX       : integer := CLK_FREQ_HZ -
            1;
49      signal one_hz_div             : integer range 0 to
            ONE_HZ_DIV_MAX := 0;
50      signal tick_1s                : STD_LOGIC := '0';
51
52      -- Multiplexado ~1 kHz por digito (8 digitos ? 8 kHz
            total)
53      constant DIGITOS               : integer := 8;
54      constant REFRESH_PER_DIGIT_HZ : integer := 1000;
55      constant REFRESH_DIV_MAX      : integer := (CLK_FREQ_HZ /
             (REFRESH_PER_DIGIT_HZ * DIGITOS)) - 1; -- ~12500-1
56      signal refresh_div            : integer range 0 to
            REFRESH_DIV_MAX := 0;
57      signal mux_idx                : unsigned(2 downto 0) := (
            others => '0');  -- 0..7
58
59      -- Buffer de digitos (0..9; 10=blank)
60      type digit_array is array (0 to 7) of integer range 0 to
            10;
61      signal digits : digit_array := (others => 10); -- 10 =
            blanco
62
63      -- ======= Decodificador 7-seg activo en bajo (CA..CG)
            =======
64      -- OJO: valida que el orden de pines CA..CG de tu placa
            coincida con este mapa.
65      function bcd_to_7seg_active_low(n: integer) return
            STD_LOGIC_VECTOR is
66          -- 0=>"0000001" 1=>"1001111" 2=>"0010010"
               3=>"0000110"
67          -- 4=>"1001100" 5=>"0100100" 6=>"0100000"
               7=>"0001111"
68          -- 8=>"0000000" 9=>"0000100" blank=>"1111111"
69          variable segs : STD_LOGIC_VECTOR(6 downto 0);
70      begin
71          case n is
72              when 0  => segs := "0000001";
```

```
73              when 1  => segs := "1001111";
74              when 2  => segs := "0010010";
75              when 3  => segs := "0000110";
76              when 4  => segs := "1001100";
77              when 5  => segs := "0100100";
78              when 6  => segs := "0100000";
79              when 7  => segs := "0001111";
80              when 8  => segs := "0000000";
81              when 9  => segs := "0000100";
82              when others => segs := "1111111";
83          end case;
84          return segs;
85      end function;
86
87      -- ======= Sincronizacion / debounce / one-shot del boton
            =======
88      constant DEBOUNCE_TICKS : integer := CLK_FREQ_HZ / 200;
            -- ~5 ms
89      signal btn_sync0, btn_sync1 : STD_LOGIC := '0';
90      signal btn_stable           : STD_LOGIC := '0';
91      signal btn_stable_prev      : STD_LOGIC := '0';
92      signal btn_pulse            : STD_LOGIC := '0';
93      signal db_cnt               : integer range 0 to
            DEBOUNCE_TICKS := 0;
94
95  begin
96      -- ======= Generador de tick 1 Hz =======
97      process(clk)
98      begin
99          if rising_edge(clk) then
100             if one_hz_div = ONE_HZ_DIV_MAX then
101                 one_hz_div <= 0;
102                 tick_1s    <= '1';
103             else
104                 one_hz_div <= one_hz_div + 1;
105                 tick_1s    <= '0';
106             end if;
107         end if;
108     end process;
109
110     -- ======= Sincronizar boton a clk (2 FFs) =======
111     process(clk)
112     begin
113         if rising_edge(clk) then
114             btn_sync0 <= btn_dec5;
115             btn_sync1 <= btn_sync0;
116         end if;
117     end process;
118
```

```
119     -- ======= Debounce por tiempo =======
120     process(clk)
121     begin
122         if rising_edge(clk) then
123             if btn_sync1 = btn_stable then
124                 db_cnt <= 0; -- sin cambio
125             else
126                 if db_cnt = DEBOUNCE_TICKS then
127                     btn_stable <= btn_sync1; -- aceptar nuevo
                            nivel
128                     db_cnt <= 0;
129                 else
130                     db_cnt <= db_cnt + 1;
131                 end if;
132             end if;
133         end if;
134     end process;
135 
136     -- ======= One-shot en flanco ascendente del boton (
            despues del debounce) =======
137     process(clk, reset)
138     begin
139         if reset = '1' then
140             btn_stable_prev <= '0';
141             btn_pulse       <= '0';
142         elsif rising_edge(clk) then
143             btn_pulse <= '0';
144             if (btn_stable = '1') and (btn_stable_prev = '0')
                        then
145                 btn_pulse <= '1';
146             end if;
147             btn_stable_prev <= btn_stable;
148         end if;
149     end process;
150 
151     -- ======= FSM + contador (prioridad al boton) =======
152     process(clk, reset)
153     begin
154         if reset = '1' then
155             current_state <= NS_VERDE;
156             counter       <= 10;
157         elsif rising_edge(clk) then
158             -- 1) Boton: restar 5s o saltar estado si quedan
                    <5s
159             if btn_pulse = '1' then
160                 if counter >= 5 then
161                     counter <= counter - 5;
162                 else
163                     current_state <= next_state;
```

```
164                     counter       <= 10;
165                 end if;
166
167             -- 2) Tictac de 1 Hz
168             elsif tick_1s = '1' then
169                 if counter > 0 then
170                     counter <= counter - 1;
171                 else
172                     current_state <= next_state;
173                     counter       <= 10;
174                 end if;
175             end if;
176         end if;
177     end process;
178
179     -- ======= Siguiente estado =======
180     process(current_state)
181     begin
182         case current_state is
183             when NS_VERDE     => next_state <= NS_AMARILLO;
184             when NS_AMARILLO  => next_state <= EO_VERDE;
185             when EO_VERDE     => next_state <= EO_AMARILLO;
186             when EO_AMARILLO  => next_state <= NS_VERDE;
187         end case;
188     end process;
189
190     -- ======= LEDs segun estado (duplicados por eje) =======
191     process(current_state)
192         -- senales intermedias por eje
193         variable ns_r, ns_y, ns_g : STD_LOGIC := '0';
194         variable eo_r, eo_y, eo_g : STD_LOGIC := '0';
195     begin
196         -- por defecto apagado
197         ns_r := '0'; ns_y := '0'; ns_g := '0';
198         eo_r := '0'; eo_y := '0'; eo_g := '0';
199
200         case current_state is
201             when NS_VERDE =>
202                 ns_g := '1';      -- N/S verde
203                 eo_r := '1';      -- E/O rojo
204             when NS_AMARILLO =>
205                 ns_y := '1';      -- N/S amarillo
206                 eo_r := '1';      -- E/O rojo
207             when EO_VERDE =>
208                 eo_g := '1';      -- E/O verde
209                 ns_r := '1';      -- N/S rojo
210             when EO_AMARILLO =>
211                 eo_y := '1';      -- E/O amarillo
212                 ns_r := '1';      -- N/S rojo
```

```
213         end case;
214
215         -- Duplicacion: N y S iguales; E y O iguales
216         led_n_rojo     <= ns_r;  led_s_rojo     <= ns_r;
217         led_n_amarillo <= ns_y;  led_s_amarillo <= ns_y;
218         led_n_verde    <= ns_g;  led_s_verde    <= ns_g;
219
220         led_e_rojo     <= eo_r;  led_o_rojo     <= eo_r;
221         led_e_amarillo <= eo_y;  led_o_amarillo <= eo_y;
222         led_e_verde    <= eo_g;  led_o_verde    <= eo_g;
223     end process;
224
225     -- ======= Calculo de digitos a mostrar (dos digitos,
        resto en blanco) =======
226     process(counter)
227         variable d, u : integer;
228     begin
229         d := counter / 10;
230         u := counter mod 10;
231
232         digits <= (others => 10); -- blanco
233         digits(0) <= u;           -- AN0 = unidades
234         digits(1) <= d;           -- AN1 = decenas
235     end process;
236
237     -- ======= Multiplexado de 8 digitos =======
238     -- divisor de refresco
239     process(clk)
240     begin
241         if rising_edge(clk) then
242             if refresh_div = REFRESH_DIV_MAX then
243                 refresh_div <= 0;
244                 mux_idx     <= mux_idx + 1; -- 0..7
245             else
246                 refresh_div <= refresh_div + 1;
247             end if;
248         end if;
249     end process;
250
251     -- seleccion de digito y anodos (activo en bajo)
252     process(mux_idx, digits)
253         variable an_v : STD_LOGIC_VECTOR(7 downto 0);
254         variable seg_v: STD_LOGIC_VECTOR(6 downto 0);
255         variable idx  : integer := to_integer(mux_idx);
256     begin
257         an_v := (others => '1');  -- todos apagados
258         an_v(idx) := '0';         -- habilita uno (activo en
            bajo)
259
```

```
260         seg_v := bcd_to_7seg_active_low(digits(idx));
261
262         an  <= an_v;
263         seg <= seg_v;
264         -- dp eliminado
265     end process;
266
267 end Behavioral;
```

Listing 1: Controlador de semáforo con cuenta regresiva y multiplexado de 7 segmentos.

4.1. Entidad y puertos

```
1  entity semaforo_dia is
2    Port (
3      clk         : in  STD_LOGIC;                 -- 100 MHz Nexys A7
4      reset       : in  STD_LOGIC;                 -- Activo en alto
5      btn_dec5    : in  STD_LOGIC;                 -- Bot n : restar 5 s / saltar estado
6
7      -- 7-seg multiplexados (activo en bajo)
8      seg         : out STD_LOGIC_VECTOR(0 to 6); -- CA..CG
9      an          : out STD_LOGIC_VECTOR(7 downto 0); -- AN7..AN0 (activo en bajo)
10
11     -- LEDs Norte
12     led_n_rojo,     led_n_amarillo,     led_n_verde    : out STD_LOGIC;
13     -- LEDs Sur
14     led_s_rojo,     led_s_amarillo,     led_s_verde    : out STD_LOGIC;
15     -- LEDs Este
16     led_e_rojo,     led_e_amarillo,     led_e_verde    : out STD_LOGIC;
17     -- LEDs Oeste
18     led_o_rojo,     led_o_amarillo,     led_o_verde    : out STD_LOGIC
19   );
20 end semaforo_dia;
```

Listing 2: Entidad y puertos del módulo

Idea clave. Los displays y los ánodos son *activos en bajo*, por lo que un '0' enciende el segmento o el dígito. Los cuatro ejes (N, S, E, O) se controlan con salidas dedicadas, pero se agrupan en pares por eje (N/S y E/O) en la lógica interna.

4.2. Tipos de datos y estados de la FSM

```
type state_type is (NS_VERDE, NS_AMARILLO, EO_VERDE,
   EO_AMARILLO);
signal current_state, next_state : state_type :=
   NS_VERDE;

signal counter : integer range 0 to 10 := 10;
```

Listing 3: Tipos y registros de la máquina de estados

Didáctica FSM. Modelo de tipo **Moore**: las salidas (LEDs) dependen únicamente del `current_state`. El ciclo es: `NS_VERDE` → `NS_AMARILLO` → `EO_VERDE` → `EO_AMARILLO` → `NS_VERDE`.

4.3. Divisores de reloj y generación de 1 Hz

```
constant CLK_FREQ_HZ          : integer := 100_000_000;

-- Tick 1 Hz
constant ONE_HZ_DIV_MAX       : integer := CLK_FREQ_HZ -
    1;
signal one_hz_div             : integer range 0 to
   ONE_HZ_DIV_MAX := 0;
signal tick_1s                : STD_LOGIC := '0';

process(clk)
begin
  if rising_edge(clk) then
    if one_hz_div = ONE_HZ_DIV_MAX then
      one_hz_div <= 0;
      tick_1s    <= '1';
    else
      one_hz_div <= one_hz_div + 1;
      tick_1s    <= '0';
    end if;
  end if;
end process;
```

```
20
21 -- Multiplexado ~1 kHz por d gito (8 d gitos -> ~8 kHz
       total)
22 constant DIGITOS                : integer := 8;
23 constant REFRESH_PER_DIGIT_HZ : integer := 1000;
24 constant REFRESH_DIV_MAX       : integer :=
25   (CLK_FREQ_HZ / (REFRESH_PER_DIGIT_HZ * DIGITOS)) - 1;
26
27 signal refresh_div : integer range 0 to REFRESH_DIV_MAX
       := 0;
28 signal mux_idx     : unsigned(2 downto 0) := (others =>
       '0');
29
30 process(clk)
31 begin
32   if rising_edge(clk) then
33     if refresh_div = REFRESH_DIV_MAX then
34       refresh_div <= 0;
35       mux_idx     <= mux_idx + 1;
36     else
37       refresh_div <= refresh_div + 1;
38     end if;
39   end if;
40 end process;
```

Listing 4: Tick de 1 Hz y divisor de refresco para multiplexado

Qué aprender. El pulso `tick_1s` (un ciclo por segundo) es la base del temporizador del semáforo. El índice `mux_idx` recorre 0..7 para seleccionar cada dígito del display sin parpadeo visible.

4.4. Sincronización, *debounce* y *one-shot* de `btn_dec5`

```
1 -- Sincronizaci n a 'clk'
2 process(clk)
3 begin
4   if rising_edge(clk) then
5     btn_sync0 <= btn_dec5;
6     btn_sync1 <= btn_sync0;
7   end if;
8 end process;
9
10 -- Debounce a ~5 ms
```

```
11 constant DEBOUNCE_TICKS : integer := CLK_FREQ_HZ / 200;
12 signal btn_stable, btn_stable_prev, btn_pulse :
      STD_LOGIC := '0';
13 signal db_cnt : integer range 0 to DEBOUNCE_TICKS := 0;
14
15 process(clk)
16 begin
17   if rising_edge(clk) then
18     if btn_sync1 = btn_stable then
19       db_cnt <= 0;
20     else
21       if db_cnt = DEBOUNCE_TICKS then
22         btn_stable <= btn_sync1;
23         db_cnt <= 0;
24       else
25         db_cnt <= db_cnt + 1;
26       end if;
27     end if;
28   end if;
29 end process;
30
31 -- One-shot
32 process(clk, reset)
33 begin
34   if reset = '1' then
35     btn_stable_prev <= '0';
36     btn_pulse       <= '0';
37   elsif rising_edge(clk) then
38     btn_pulse <= '0';
39     if (btn_stable = '1') and (btn_stable_prev = '0')
          then
40       btn_pulse <= '1';
41     end if;
42     btn_stable_prev <= btn_stable;
43   end if;
44 end process;
```

Listing 5: Pipeline de entrada: sincronización, antirrebote y pulso único

Patrón universal. Toda entrada asíncrona debe: (1) sincronizarse, (2) desrebotarse, (3) convertir flancos en pulsos de un ciclo. Esto evita metaestabilidad y múltiples disparos indeseados.

4.5. Registro de estado y contador con prioridad al botón

```
process(clk, reset)
begin
  if reset = '1' then
    current_state <= NS_VERDE;
    counter       <= 10;

  elsif rising_edge(clk) then
    -- 1) Bot n: restar 5s o saltar estado si quedan <5
       s
    if btn_pulse = '1' then
      if counter >= 5 then
        counter <= counter - 5;
      else
        current_state <= next_state;
        counter       <= 10;
      end if;

    -- 2) Tictac de 1 Hz
    elsif tick_1s = '1' then
      if counter > 0 then
        counter <= counter - 1;
      else
        current_state <= next_state;
        counter       <= 10;
      end if;
    end if;
  end if;
end process;
```

Listing 6: Secuencial: prioridad al botón, luego al tick

Lección. Este proceso es el *registro de estado* canónico de una FSM: solo cambia en flancos de reloj o `reset`. La prioridad al botón define un comportamiento determinista cuando el usuario interactúa.

4.6. Lógica de transición (`next_state`)

```
process(current_state)
begin
  case current_state is
    when NS_VERDE     => next_state <= NS_AMARILLO;
    when NS_AMARILLO  => next_state <= EO_VERDE;
    when EO_VERDE     => next_state <= EO_AMARILLO;
    when EO_AMARILLO  => next_state <= NS_VERDE;
  end case;
end process;
```

Listing 7: Combinacional: cálculo de `next_state`

FSM en tres procesos. Separar (1) registro de estado, (2) siguiente estado y (3) salidas simplifica verificación, síntesis y reutilización. El alumno debe adoptar este patrón como *best practice*.

4.7. Asignación de salidas (LEDs por eje)

```
process(current_state)
  variable ns_r, ns_y, ns_g : STD_LOGIC := '0';
  variable eo_r, eo_y, eo_g : STD_LOGIC := '0';
begin
  ns_r := '0'; ns_y := '0'; ns_g := '0';
  eo_r := '0'; eo_y := '0'; eo_g := '0';
  case current_state is
    when NS_VERDE     => ns_g := '1'; eo_r := '1';
    when NS_AMARILLO  => ns_y := '1'; eo_r := '1';
    when EO_VERDE     => eo_g := '1'; ns_r := '1';
    when EO_AMARILLO  => eo_y := '1'; ns_r := '1';
  end case;

  -- Duplicaci n por eje
  led_n_rojo     <= ns_r;  led_s_rojo     <= ns_r;
  led_n_amarillo <= ns_y;  led_s_amarillo <= ns_y;
  led_n_verde    <= ns_g;  led_s_verde    <= ns_g;

  led_e_rojo     <= eo_r;  led_o_rojo     <= eo_r;
  led_e_amarillo <= eo_y;  led_o_amarillo <= eo_y;
  led_e_verde    <= eo_g;  led_o_verde    <= eo_g;
end process;
```

Listing 8: Combinacional: salidas tipo Moore para LEDs

Claves. Las salidas dependen solo del estado actual (Moore). La duplicación N/S y E/O garantiza consistencia física de los dos semáforos por eje.

4.8. Decodificador BCD a 7 segmentos (activo en bajo)

```
function bcd_to_7seg_active_low(n: integer) return
   STD_LOGIC_VECTOR is
  -- 0=>"0000001" ... 9=>"0000100", blank=>"1111111"
  variable segs : STD_LOGIC_VECTOR(6 downto 0);
begin
  case n is
    when 0  => segs := "0000001";
    when 1  => segs := "1001111";
    when 2  => segs := "0010010";
    when 3  => segs := "0000110";
    when 4  => segs := "1001100";
    when 5  => segs := "0100100";
    when 6  => segs := "0100000";
    when 7  => segs := "0001111";
    when 8  => segs := "0000000";
    when 9  => segs := "0000100";
    when others => segs := "1111111";
  end case;
  return segs;
end function;
```

Listing 9: Función BCD a 7-seg activo en bajo

Nota. Verifica el mapeo `CA..CG` de tu placa: si el orden físico difiere, ajusta los bits del patrón. Mantén el convenio `(6 downto 0)` también en el puerto `seg` para coherencia.

4.9. Cálculo de dígitos (decenas y unidades)

```
type digit_array is array (0 to 7) of integer range 0 to
    10;
signal digits : digit_array := (others => 10); -- 10 =
   blanco

process(counter)
  variable d, u : integer;
begin
  d := counter / 10;
  u := counter mod 10;

  digits <= (others => 10); -- blanco
  digits(0) <= u;           -- AN0 = unidades
  digits(1) <= d;           -- AN1 = decenas
end process;
```

Listing 10: Buffer de dígitos y partición decenas/unidades

Buena práctica. Los dígitos no usados quedan en "blanco" (1111111). Si deseas compatibilidad con 4 dígitos, reserva índices y añade símbolos (p.ej., guion cuando `counter` sea negativo en otras prácticas).

4.10. Multiplexado de 8 dígitos

```
process(mux_idx, digits)
  variable an_v : STD_LOGIC_VECTOR(7 downto 0);
  variable seg_v: STD_LOGIC_VECTOR(6 downto 0);
  variable idx  : integer := to_integer(mux_idx);
begin
  an_v := (others => '1');    -- todos apagados (activo
     en bajo)
  an_v(idx) := '0';           -- habilita uno

  seg_v := bcd_to_7seg_active_low(digits(idx));

  an  <= an_v;
  seg <= seg_v;
end process;
```

Listing 11: Selección de dígito y actualización de ánodos/segmentos

Concepto. `mux_idx` selecciona el dígito activo y el decodificador determina el patrón de segmentos. La frecuencia efectiva es suficientemente alta para eliminar parpadeo.

4.11. Patrón universal para cualquier máquina de estados

Para diseñar **cualquier** FSM en VHDL, sigue este esqueleto de tres procesos:

1. **Registro de estado** (*secuencial*): actualiza `current_state` en flanco de reloj y gestiona `reset`.
2. **Próximo estado** (*combinacional*): calcula `next_state` a partir de `current_state` y entradas sincronizadas.
3. **Salidas** (*combinacional*): define señales de salida solo en función del estado (Moore) o del estado y entradas (Mealy).

Este patrón, junto con el pipeline de entradas (sincronización + debounce + one-shot) y una temporización basada en ticks, permite construir FSMs robustas y escalables para cualquier problema de control digital.

4.12. Diagrama de bloques

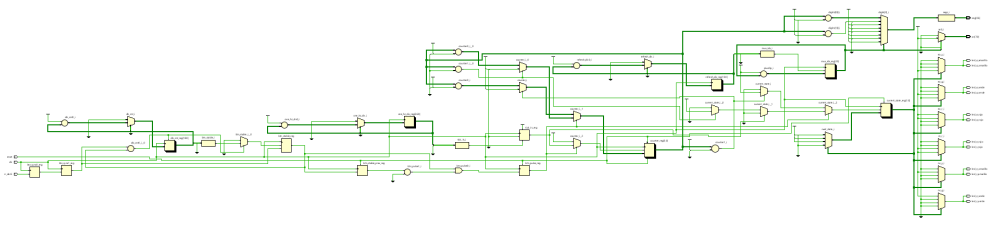


Figura 5: Diagrama de bloques: divisores de reloj, tratamiento de botón, FSM, lógica de salidas y subsistema de visualización (decodificador + multiplexado).

4.13. Mapeo de pines y niveles eléctricos (Nexys A7)

En esta subsección se presenta el **mapeo lógico–físico** de todas las señales del diseño hacia pines del FPGA. Las tablas están organizadas por

función: control/temporización, displays de 7 segmentos y salidas a LEDs por eje. Todas las E/S usan `LVCMOS33`.

Convenciones clave.

- Los ánodos del display (`an[7:0]`) y los segmentos (`seg[6:0]`) son **activos en bajo**.
- Para mantener coherencia con el decodificador, adopta la convención `seg(6 downto 0)` ≡ `CA..CG`. Si tu entidad declara `seg(0 to 6)`, homogeneiza direcciones o ajusta el orden de bits en el tope.
- `reset` es activo en alto; `btn_dec5` requiere sincronización+*debounce* (ya implementados en el código).

Cuadro 1: Señales de control y temporización

Señal (VHDL)	**Dirección**	**Pin (Package)**	**IOSTANDARD**
`clk`	Entrada	E3	LVCMOS33
`reset`	Entrada (act. alto)	J15	LVCMOS33
`btn_dec5`	Entrada	M17	LVCMOS33

Cuadro 2: Display de 7 segmentos: ánodos (activos en bajo)

Dígito	`an[7]`	`an[6]`	`an[5]`	`an[4]`	`an[3]`	`an[2]`
Pin	U13	K2	T14	P14	J14	T9
Dígito	`an[1]`	`an[0]`	(resto no usado se mantiene apagado)			
Pin	J18	J17				

Cuadro 3: Display de 7 segmentos: segmentos (activos en bajo)

Bit	`seg[6]`	`seg[5]`	`seg[4]`	`seg[3]`	`seg[2]`	`seg[1]`
Pin	L18	T11	P15	K13	K16	R10
Bit	`seg[0]`	(convención sugerida: `seg(6..0)` ≡ CA..CG)				
Pin	T10					

Cuadro 4: Salidas a LEDs por eje

Señal (VHDL)	Color	Pin (Package)	IOSTANDARD
`led_n_rojo`	Rojo	F16	LVCMOS33
`led_n_amarillo`	Amarillo	D14	LVCMOS33
`led_n_verde`	Verde	G16	LVCMOS33
`led_s_rojo`	Rojo	E17	LVCMOS33
`led_s_amarillo`	Amarillo	D17	LVCMOS33
`led_s_verde`	Verde	F18	LVCMOS33
`led_e_rojo`	Rojo	E18	LVCMOS33
`led_e_amarillo`	Amarillo	D18	LVCMOS33
`led_e_verde`	Verde	G17	LVCMOS33
`led_o_rojo`	Rojo	F13	LVCMOS33
`led_o_amarillo`	Amarillo	E16	LVCMOS33
`led_o_verde`	Verde	G13	LVCMOS33

5. Resultados en la placa Nexys

Una vez implementado y cargado el diseño en la tarjeta Nexys A7, se procede a la verificación de su funcionamiento. Para ello se utilizaron los recursos del display de 7 segmentos y el switch de reset conectado al pin J15, de acuerdo con la asignación de pines previamente definida.

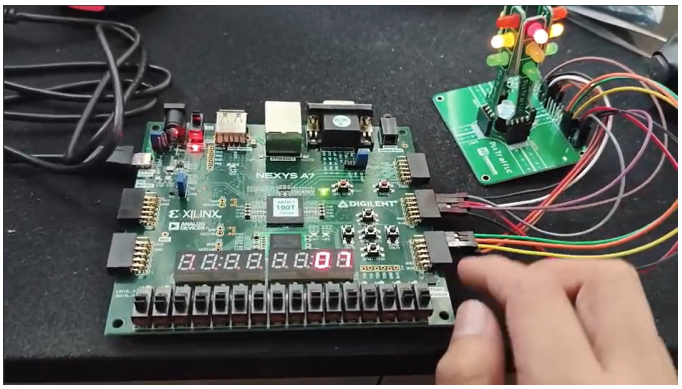


Figura 6: Visualización del semaforo con contador en la Nexys.